Fake It till You Make It: Enhancing Security of Bluetooth Secure Connections via Deferrable Authentication

Marc Fischlin
Cryptoplexity
Technische Universität Darmstadt
Darmstadt, Germany
marc.fischlin@tu-darmstadt.de

Olga Sanina
Cryptoplexity
Technische Universität Darmstadt
Darmstadt, Germany
olga.sanina@tu-darmstadt.de

Abstract

The Bluetooth protocol for wireless connection between devices comes with several security measures to protect confidentiality and integrity of data. At the heart of these security protocols lies the Secure Simple Pairing, wherewith the devices can negotiate a shared key before communicating sensitive data. Despite the good intentions, the Bluetooth security protocol has repeatedly been shown to be vulnerable, especially with regard to active attacks on the Secure Simple Pairing.

We propose here a mechanism to limit active attacks on the Secure Connections protocol (the more secure version of the Secure Simple Pairing protocol), without infringing on the current Bluetooth protocol stack specification. The idea is to run an authentication protocol, like a classical challenge-response step for certified keys, within the existing infrastructure, even at a later, more convenient point in time. We prove that not only does this authentication step ensure freshness of future encryption keys, but an interesting feature is that it—*a posteriori*—also guarantees security of previously derived encryption keys. We next argue that this approach indeed prevents a large set of known attacks on the Bluetooth protocol.

CCS Concepts

• **Security and privacy → Cryptography**.

Keywords

Bluetooth; key exchange; secure connections; authentication; deferrable outside first use (DOFU)

ACM Reference Format:
Marc Fischlin and Olga Sanina. 2024. Fake It till You Make It: Enhancing Security of Bluetooth Secure Connections via Deferrable Authentication. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24), October 14–18, 2024, Salt Lake City, UT, USA*. ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3658644.3670360

CCS '24, October 14–18, 2024, Salt Lake City, UT, USA

ACM ISBN 979-8-4007-0636-3/24/10
https://doi.org/10.1145/3658644.3670360

1 Introduction

Bluetooth enables short-ranged wireless communication between and for users. Predicted by ABI Research to incline the shipment into the market in the next 5 years, Bluetooth technology is already used in billions of devices in smart homes, cars, monitoring and control systems, and wearables. With this spread, Bluetooth channels send more and more data, including sensitive medical information, hence data confidentiality is of concern. This includes authentication that ensures only intended devices can be connected.

Bluetooth was invented in the '90s and, naturally, has evolved over time. Each major change is depicted as a version in a Core Specification maintained by the Bluetooth Special Interest Group (SIG). The latest version 5.4 [8] was introduced on the 31st of January 2023, and adds, to give an example, the use of electronic price tags. The modifications resulted in different versions of the Bluetooth protocol suits, which include (outdated) legacy protocols and are available as two major branches today: classical Bluetooth BR/EDR and low-energy Bluetooth BLE. All the so-called authentication methods suggested in the Bluetooth Core Specification are not secure and prone to numerous attacks, also very recent ones.[1]

1.1 Bluetooth's Etiology

To protect the communication, Bluetooth Core Specifications cover various cryptographic mechanisms. At foremost, starting with version 4.2, BR/EDR and BLE support Secure Connections (SC), which should enable parties to communicate securely, based on approved cryptographic primitives such as AES and HMAC.

The core of SC is the Secure Simple Pairing (SSP) protocol that allows establishment of a shared key between the devices and is based on the elliptic-curve Diffie–Hellman (ECDH) protocol. To obtain some form of authentication, devices rely on the input/output capabilities (*IOcap*) that they negotiate during the pairing. This includes the association models NC (numeric comparison, users comparing displayed digits), PKE (passkey entry, users entering digits), and OOB (out-of-band transfer of authentication data without the direct user involvement into the protocol). However, the protocol may also run in JW (just works) without authentication, user involvement, and dependency on *IOcap*. The concrete instantiation of SSP slightly differs between BR/EDR and BLE.

From the link key (BR/EDR) resp. long-term key (BLE), created in the SSP protocol, the parties can then derive encryption keys, which can be used to secure the actual communication. Reconnections also use this mechanism to create fresh encryption keys from the link key resp. long-term key, avoiding the need to pair again.

[1]The reported attacks can be found at: https://www.bluetooth.com/learn-about-bluetooth/key-attributes/bluetooth-security/reporting-security/

Although Bluetooth incorporates cryptographic mechanisms, many security design choices are not motivated well and appear to be ad hoc. Indeed, the history of the Bluetooth security protocol suite abounds with devastating attacks and thorny security analyses. Most of the attacks against Bluetooth focus on the SSP protocol and the authentication property. This includes attacks like the downgrading to JW attack [16], enforcing that intended authentication is omitted; the Method Confusion Attack [32] and the Pairing Confusion Attack [11], in which the adversary unnoticedly modifies the IO capabilities in the pairing step and thus changes the authentication method. The Ghost Keystroke Attacks [19, 37] exploit cross-connected executions, which leak passkeys used for authentication to MitM adversaries. Other attacks decrease the strength of the encryption scheme by downgrading the keysize, such as the KNOB Attack [3, 5] and the Keysize Confusion Attack [27]. They can be used in combination with authentication attacks, like the BIAS [2] and BLUFFS [1] Attacks, to cause even more dramatic consequences. For legacy versions of Bluetooth, further attacks aim at the possibility of using brute-forceable low-entropic secrets [18, 25]. Some attacks use the protocol support to switch between the BR/EDR and BLE modes, including the BLURtooth attack [4] and the CSIA attack [35]. The BLESA attack [33] against reconnections in BLE exploits the lack of proper authentication. Table 1 gives an overview on the effectiveness of some suggested countermeasures [27, 31] against various attacks. Note that our remedy works within the Secure Connections type on the application level and thus cannot thwart attacks on the Legacy protocols. To highlight the efficacy of our measure, we discuss the Ghost Keystroke attack and its mitigation more closely in Section 8; all other attacks appear in the full version of the paper [14].

Table 1: Proposed patches and their resilience to the attacks. Symbols: ✗ means the attack is possible, ✓ means the attack is prevented, (✓) is not claimed but prevented. All attacks are possible for both BLE and BR/EDR transport, apart from [25] resp. [1, 2, 18, 22] that affect only the BLE resp. BR/EDR transport. Note that our approach cannot prevent attacks on the Legacy versions.

Attack	Exploited Stage	Solution		
		[31]	[27]	Ours
Role Confusion [31]	PKE	✓	(✓)	✓
Method Confusion [32]	NC, PKE	✗	✓	✓
Pairing Confusion [11]	NC, PKE, Legacy	✗	✗	✓
Ghost Keystroke [19, 37]	PKE, (NC)	✗	✗	✓
Downgrade to JW [15, 16]	SC and SSP	✗	✗	✓
KNOB [3, 5, 27]	Key Size Negotiation	✗	✗	✗
Key Cracking [18, 25]	Legacy	✗	✗	✗
BIAS [2]	Reconnection	✗	✗	✗
BLUFFS [1]	Legacy Reconnection	✗	✗	✗
BLURtooth [4], CSIA [35]	Cross-transport	✗	✗	✗

1.2 Contributions

Our contribution enhances the security of the existing Bluetooth Secure Connections protocol stack in a backward-compatible manner and consists of the three main parts.

First, unlike the usually suggested fixes for Bluetooth, we propose a (cryptographic) method to thwart many attacks (see the right column in Table 1). Based on the observation that most vulnerabilities root in the lack or misuse of message and identity authentication in association models, we propose an authentication protocol that builds on the established paradigm of challenge-response schemes. This protocol is executed on top of (i.e., after) the current steps, ensuring that the communication partner is authentic and uses the same link/long-term key. Such a solution requires some form of authentication means, like certified signing keys linked to the Bluetooth address of the device. We propose different variants of authentication protocols but our method only yields a provably secure solution for BR/EDR.[2]

Our protocols blend in well with the existing Bluetooth protocol stack: We rely on the defined cryptographic functions supported by the Core Specification; the protocols can run as an application over the Bluetooth channel and they do not require access to secrets or other security-relevant data beyond the link/long-term keys. The authentication step can be executed later, at any point in time, and still ensures the security of the keys "backwards". It means that, upon successful completion of the authentication part, one can deduce that previous encryption keys must have been secure. This feature differs from perfect forward secrecy (PFS), which guarantees the previous session keys to remain secure if the adversary compromises the long-term secret. For PFS, the keys are usually secure from the beginning; in contrast, our security property says that authentication transfers the keys with the unknown status to secure keys. While in both cases something happens at a later point in time (compromise vs. authentication), the consequences are different: the keys remain secure after the compromise vs. they are made secure against active attacks.

Post-handshake protocols have been considered in the literature before, e.g., for the Internet Key Exchange protocol IKEv2 [20] in [26] or for TLS 1.3 in [17] and [21]. The difference to our setting here is that these authentication steps are usually executed immediately after the key establishment, assuming that the key establishment data (e.g., communication transcript) is still available. For Bluetooth communication, the authentication process may run much later: either as a part of the application execution, or after a software update (e.g., installing the authentication data), or because the battery was low during the pairing procedure—but no fresh pairing is needed in these cases anymore. We discuss the relationship to existing approaches in more detail in Section 4.

While our solution complies with the protocol flow of Bluetooth, it requires some form of certification of identities and keys, e.g., in the form of a certificate issued by trustworthy vendors. The vendors themselves may be certified with a root certificate held by the Bluetooth SIG. Such certificates for vendors could, for example, be issued as part of the mandatory *Bluetooth Qualification Process* for new products. This requires additional administrative steps but is conceivably easier to integrate than making changes at the level of the specification and obeying legacy compatibility.

[2]The reason is that link keys in BR/EDR are derived via (truncated) HMAC and thus obey collision resistance. In contrast, BLE uses an AES-CMAC-based key derivation function, which is not known to provide the required level of robustness, as it is malleable [12]. We discuss this more in Section 1.3.

Since certification is already part of the Bluetooth ecosystem in Bluetooth Mesh (Mesh specification supports X.509 digital certificates since version 1.1 but requires OOB methods for incorporating such authentication methods), we find it plausible to assume that certification can be implemented in Bluetooth BR/EDR and LE, too, both administratively and also technically. In terms of the computational costs, Bluetooth devices already implement procedures like ECDH computations, so, e.g., EdDSA should not be more expensive than this. Furthermore, we alternatively suggest a signature-free authentication method, which also employs ECDH.

Second, we extend the trust-on-first-use (TOFU) security model for key establishment and reconnections in Bluetooth [13], which is a game-based model in the Bellare–Rogaway (BR) style [6]. While the TOFU model opts for passive adversaries during the initial connections to ensure the security of derived keys, we add authentication requirements to allow deferrable-outside-first-use (DOFU) authentication. That is, the authentication might be postponed to any later point in time and thusyields TOFU-OR-DOFU security.

Finally, we prove our proposed solution to provide an adequate level of cryptographic strength in the sense of authenticated key exchange protocols. That is, we show the new protocols enjoy the security in our extended TOFU-OR-DOFU model. Note that Fischlin and Sanina [13] showed the best result one can hope for, for the "vanilla" Bluetooth protocol stack: session keys (called encryption keys in Bluetooth) are secure as long as the adversary has been passive in the initial connection, wherein the link key (BR/EDR) resp. long-term key (BLE) has been distributed and whence all encryption keys are derived. We augment this result by stating that all encryption keys are secure if the initial pairing is trustworthy, *or if the authentication step has been carried out successfully.*

1.3 Applicability to BR/EDR and BLE

We emphasize here that our positive results in BR/EDR with HMAC as a key derivation function (KDF) for *LK* do not immediately transfer to BLE because BLE uses AES-CMAC as a KDF function for *LTK*. HMAC in BR/EDR gives collision resistance of the derived link key as an additional feature, which we require in the proof for deferrable authentication. The AES-CMAC function in BLE, however, is not collision-resistant and the attacks in [12] showed how such weaknesses can be exploited. We still managed to prove our protocols for BLE to have provided match security, authentication, and leakage resistance as long as AES-CMAC is used (but key secrecy only in the TOFU scenario). This is not a shortcoming of our proof, but rather the lack of the collision resistance of the KDF in BLE opens up the following attack strategy if only the long-term key can be used for authentication. The adversary can first make two unpartnered honest initial-connection sessions accept and hold the same long-term key *LTK* due to the lack of the collision resistance of the KDF in BLE. If the adversary connects these two sessions in subsequent authentication sessions, then authentication would succeed for the same *LTK*, although the initial-connection sessions were not partnered. This breaks the idea of the authentication intuitively and can also be exploited formally by the adversary. If the KDF in BLE was replaced by a collision-resilient variant, the proof of key secrecy would go through smoothly for BLE as well. Unfortunately, the BLE specification currently employs only AES and AES-CMAC as KDFs.

2 Notation

We briefly recap the notation used throughout the paper and in Bluetooth. The model-specific definitions are given in Section 5. Acronyms used in Bluetooth and in the paper can be found in the full version of the paper [14].

2.1 General Notation

By $\mathbb{Z}_m$ we denote the set of integers $0, \ldots, m-1$, often also view it as the group or ring with common modular addition and multiplication as operations. With $x \leftarrow\!\$\, X$ we denote a uniformly sampled element from the set X. In particular, $x \leftarrow\!\$\, \mathbb{Z}_{(q+1)/2} \setminus \{0\}$ means to uniformly pick an integer x from the range $1, \ldots, (q+1)/2$, where q is odd. We denote by g^x the x-fold multiplication of the generator g of a group. The common Diffie–Hellman share of public data g^a and g^b is thus given by g^{ab}. When using elliptic curves, as in Bluetooth, it is understood that g^a is the a-fold addition of the curve point g. In this case, we denote by $\langle g^a \rangle_x$ the x-coordinate of the curve point. For more background on elliptic curves see [28].

The notation $V \leftarrow v$ means that value v is assigned to variable V, $V := W$ means the variable V is defined to be equal to variable W, $\perp$ denotes an uninitialized value or an error. For example, for an array $A[]$, we let $A[i]$ be the ith element, where we assume $A[i] = \perp$ if the value has not been set previously. Given a vector A of named attributes $a, b, c, \ldots$ we often denote the values of the attributes as $A.a, A.b, A.c, \ldots$ or simply as $a, b, c, \ldots$ if the context A is clear. For strings s and r, we denote by $s|r$ the concatenation of the two strings, and the bit-wise XOR of equal-length strings by $s \oplus r$.

The deterministic resp. randomized output y of an algorithm $\mathcal{A}$, run with input x, is denoted by $y \leftarrow \mathcal{A}(x)$ resp. $y \leftarrow\!\$\, \mathcal{A}(x)$. We use square brackets to denote an optional input to an algorithm, e.g., $y \leftarrow\!\$\, \mathcal{A}(x, [a])$ means that algorithm $\mathcal{A}$ may receive auxiliary input a in addition to x. We write $\mathcal{A}^{\mathrm{O}_1,\ldots,\mathrm{O}_n}$ if the algorithm has access to oracles $\mathrm{O}_1, \ldots, \mathrm{O}_n$.

The security parameter is denoted by λ or, in unary, by 1^λ. If adversary $\mathcal{A}$ interacts in some security experiment Game with some scheme Π for security parameter λ, we write $\mathbf{Exp}^{\mathrm{Game}}_{\mathcal{A}}(\lambda) = 1$ to denote that the experiment outputs 1. This usually indicates an adversary's success, such that $\Pr\left[\mathbf{Exp}^{\mathrm{Game}}_{\mathcal{A}}(\lambda) = 1\right]$ represents the probability of this success. The advantage of the adversary is then denoted by $\mathbf{Adv}^{\mathrm{Game}}_{\mathcal{A},\Pi}$ and measures the adversary's success probability over trivial strategies, e.g., over the guessing probability $\frac{1}{2}$ for decisional experiments.

2.2 Bluetooth-specific Notation

Bluetooth employs a number of variables, whose notation follows the standards closely and which we present here. Each party has a Bluetooth address, which we denote by BD_ADDR or sometimes as A. Furthermore, we use the common values appearing in the pairing step, including the nonce N, random passkey r, commitment value C, verification value V, and check value E. Devices usually provide or take input and output capabilities *IOcap*, authentication requirements *AuthReq*, and out-of-band authentication data *OOB*.

The keys in Bluetooth are denoted as the link key *LK* (for BR/EDR), the long-term key *LTK* (for BLE), or either of them *L(T)K*. To derive further session-specific keys, devices use the device key *dk*, the authentication random number *AU_RAND*, the signed response *SRES*, the session key diversifier *SKD*, the initialization vector *IV*, the authentication ciphering offset *ACO*, and the encryption key $\mathsf{k}_{\mathrm{AES}}$. The latter is used to protect the actual communication.

Bluetooth often uses truncated strings, i.e., we write $s/2^n$ if the n leftmost bits of string s are taken, and $s \bmod 2^{32}$ if least significant bits of s are considered. If the string is given in hexadecimal representation, this is indicated by the prefix `0x`.

3 Bluetooth Background

Further we give a concise explanation of the main aspects of the Bluetooth technology. Today, Bluetooth exists in two main versions, low-energy (BLE) and classic (BR/EDR).[3] Both versions have similar discovery and connection mechanisms but they deviate in the technical aspects and applications, making them incompatible with each other. Devices might implement either of the versions (single-mode) or both (dual-mode). The latter may also switch the mode (transport) afterwards in reconnections.

Devices supporting BR/EDR are equipped with a unique and registered 48-bit MAC-address, called public (identity) address. BLE extends the address space by static random addresses of the same size, which are generated once upon booting up but otherwise stay fixed. For improved privacy, BLE also introduces resolvable and non-resolvable random private addresses. The former type can be used in reconnections, after the initial connection has been carried out with a static address. It can be resolved by the other party in a reconnection and mapped back internally to the static addresses via entries of already established connections.

The Bluetooth technology consists of the protocols on different layers, which are split into two components: the *host* for higher-layer aspects and the *controller* for lower-layer aspects—connected through a host-controller interface. The controller is usually harder to modify since it is closer tied to the hardware design. This is why we target the higher-layer protocols to integrate our authentication subprotocol. We emphasize that the cryptographic part is treated differently in the different versions: in BR/EDR, the Link Manager Protocol (LMP) is responsible for cryptographic components, while in BLE, cryptography is moved to Security Manager Protocol (SMP).

To connect, devices make links on several layers: physically, logically, and cryptographically. The example protocol flow for the connection in BR/EDR on a cryptographic level is shown in Figure 1. During the link establishments, devices learn the Bluetooth Address `BD_ADDR` of each other, the devices' capabilities (i.e., what algorithms they support and security they request). To establish cryptographic keys and start secure data exchange, devices need to pair. At this step, they can decide if they want to create a bond during the pairing (i.e., store the derived keys and delete all the temporal pairing information) or opt for one-time connection only. Here the devices also decide if they want to use Secure Connections Only (SCO) or agree on weaker versions of the protocol.

The basis of SCO is the Secure Simple Pairing (SSP) protocol, a Diffie–Hellman-based key exchange protocol. SSP comes in four possible association models: Out-of-Band (OOB) with the exchange of data over the secure channel outside of the Bluetooth transmission; PasskeyEntry (PKE) resp. NumericComparison (NC) with the user entering resp. comparing 6 displayed digits; and JustWorks (JW) with no attempt to protect against MitM attacks. As a result, SSP with the corresponding association model outputs the key, which is then used for the derivation of the encryption keys with the lifespan of a session. Usually the derived keys (link key *LK* for BR/EDR and long-term key *LTK* for BLE) are stored in the host and passed to the controller when the latter requests them.

The most vulnerable is the exchange of capabilities, as this stage influences the choice of the association model, encryption key size[4], and types of keys to derive. Most of the attacks happen here, e.g., modifying IO capabilities allows the adversary to launch the downgrade attack [16] to the JW association model, or the Method Confusion Attack [32], or an impersonation attack [37]; modifying OOB flag leads to the drop of the OOB association model [15]; avoiding SCO downgrades to the legacy protocol, which enables Pairing Confusion Attack [11], BIAS [2], BLUFFS [1], or attacks on the legacy version [24, 25]. Adversary can also downgrade the size of the keys [3, 5] or confuse the devices on the key size to use [27].

Since the encryption keys are valid for one session, the devices can reconnect using the stored link/long-term key and authenticate each other with this key. In BR/EDR, reconnection is a challenge-response scheme. The parties each choose a fresh nonce (challenge), exchange it, and authenticate each other with the responses, which are based on these fresh values and the stored key. In BLE, the procedure is simpler and consists of the two exchanged fresh values to which the *LTK* is applied to generate the encryption key.

Cryptographic Details. When SSP starts, the devices possess the Bluetooth addresses `BD_ADDR` as identities and the *IOcap* of each other and know what cryptographic algorithms the partner is able to use. This includes the used elliptic curves, which are FIPS-approved and given in the Core Specification. For curve points, Bluetooth makes use of the x-coordinate only (although the y-coordinate is also shared to prevent the Invalid Curve Attack [7]), which we mark as $\langle g^a \rangle_x$. Both parties initially establish a Diffie–Hellman key, then run the corresponding association model, and finally confirm the derived keys via values *Ea* and *Eb*.

Both versions of Bluetooth employ different cryptographic algorithms: AES-CMAC in BLE and SHA with HMAC in BR/EDR. To compute the commit and confirmation values, the link key, the device key (a spin-off key for authentication in reconnections), the encryption key, and the response to a challenge, BR/EDR uses HMAC; the derived DHKey is straight used as MACKey; for the computation of the displayed digits in NC, SHA-256 is directly employed. BLE uses AES-CMAC for all these cases, apart from the encryption key derivation in which simply AES encryption is used. Bluetooth usually truncates the output and uses the leftmost n bits, which we denote by $/2^n$.

[3] We neglect the Legacy protocols and the Mesh transport here.

[4] In BR/EDR, the negotiation of the encryption key size happens during the reconnection.

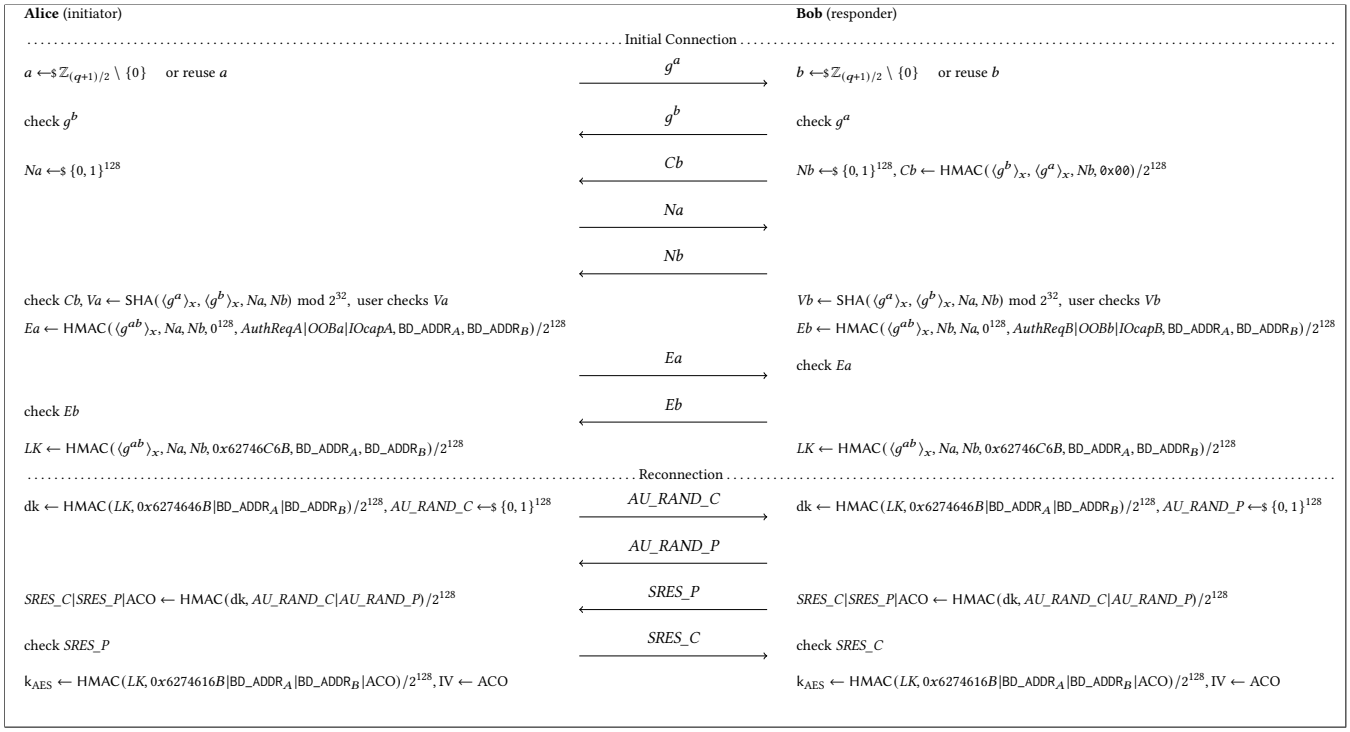

Figure 1: Bluetooth initial pairing (Secure Simple Pairing in Numeric Comparison) and reconnection (Secure Authentication and Encryption Key Derivation). The session identifier in the initial connection is $\mathsf{sid} = (\langle g^a\rangle_x, \langle g^b\rangle_x, \boldsymbol{A}, \boldsymbol{B}, \boldsymbol{Na}, \boldsymbol{Nb})$ **and in the reconnection** $\mathsf{sid} = (\boldsymbol{AU\_RAND\_C}, \boldsymbol{AU\_RAND\_P}, \boldsymbol{A}, \boldsymbol{B})$**.**

4 Related Work

In the following, we detail related works about attacks on Bluetooth and potential countermeasures, existing analyses of the protocols and cryptographic techniques related to enhancement of Bluetooth.

4.1 Attacks on Bluetooth

Cäsar et al. [9] give an overview of the attacks on BLE in both Legacy and SC. They look into the versions of the Core Specification from 4.0 to 5.2 and cover attacks that violate both security and privacy, even if they have been mitigated. Wu et al. [34] extend the overview to BR/EDR and Mesh and include the attacks on a soft- and hardware level. We refer to these papers for an overview of the vulnerabilities in Bluetooth.

Most of the attacks are mitigated by our solution (Table 1) since they result in an honest device and a compromised device deriving different *L(T)K*s in their corresponding sessions, and hence the adversary is not able to go through the authentication process if the signature or MAC is not calculated over the same keys. In the full version of the paper [14], we describe the attacks that are relevant for this paper and why other proposals patch (or not) these attacks.

4.2 Analyses of Bluetooth Security Protocols

Bluetooth was also a target of several security analyses. While some results purely focused on single protocols, others tried to consider the whole protocol stack. Both security proofs and automated tools were employed for the analysis.

Complexity-based Analyses. Lindell has analyzed the NC association model in [23] and showed NC to be secure as comparison-based key exchange, notion of which is also introduced in that paper. The analysis of the protocol is done in isolation and assumes that the Bluetooth addresses (which are not constant identities) are included in the confirmation value computation. Sun and Sun [30] have also analyzed NC and OOB in isolation and showed their security. Their result is the same as [23], but the model is more restricted, as they do not allowing the adversary to communicate with the parties after testing.

Troncoso and Hale [31] have developed a model for the analysis of protocols with user interaction. The model, called CYBORG, captures an adversarial influence on the User-to-Device (UtD) channel. They analyze different cases (which depend on the *IOcap* of the devices) in the PKE association model of Bluetooth SSP and show that PKE does not provide security in the CYBORG model: in the case of the user-generated passkey, the adversary can perform a role confusion attack and make the sessions accept with role disagreement; in case of the device-generated passkey, the adversary is able to perform a guessing attack and win with non-negligible probability. Troncoso and Hale do not provide a detailed analysis of NC subprotocol, but conclude it does not achieve the security with respect to the CYBORG model.

Yin [36] has analyzed the OOB and NC association models in terms of the CYBORG security model. The analysis showed only bidirectional OOB to be CYBORG-secure: For NC, they found an

attack where the adversary successfully substitutes the user's response to the device from negative to positive; in unidirectional OOB, they allowed the adversary to modify the data in the OOB channel and showed a successful attack in this setting. Yin has also extended the CYBORG model to address Tap'n'Ghost attacks, where the channel is authenticated but not confidential, and called it Authentication Protocols assisted by EXtra (APEX) model. They showed the APEX-security of NC and bidirectional OOB but presented an attack on the improved PKE from Troncoso and Hale [31] with the leaked passkey and unidirectional OOB (where the non-authenticated party is impersonated). Both scenarios are not intended to be secure according to the Bluetooth Core Specification. The analysis considered all the protocols in isolation, what does not represent the real-world scenario and ignores the known downgrade and confusion attacks on Bluetooth, such as [11, 16, 32].

Fischlin and Sanina [13] have analyzed all Secure Connections protocols together (i.e., excluding the Legacy protocols) and showed the security of Bluetooth in the trust-on-first-use (TOFU) model, where the adversary can eavesdrop on the initial connection and be active during the reconnections. While that analysis opted for the unmodified "vanilla" protocol and showed the best security the current version can achieve, this paper aims to improve the Bluetooth security guarantees by modifying the protocol in a backward-compatible manner.

Formal Analyses. Chang and Shmatikov [10] have conducted the first formal analysis on Bluetooth: namely, the BR/EDR Legacy protocol and the NC association model of the Bluetooth Simple Pairing protocol (the predecessor of SSP). They have used ProVerif for the analysis and modeled the user as a human oracle. The analysis confirmed the insecurity of BR/EDR Legacy and found a vulnerability that results in the difficulty for the user to distinguish for which concurrent sessions the numbers are being matched. As a solution, the authors suggest adding session identificators into the displayed digits' computation.

Wu et al. [33] used ProVerif to analyze the reconnection stage in BLE and found the Bluetooth Low Energy Spoofing Attacks (BLESA) that allow the adversary to spoof the attribute value from the Peripheral and use itto disable encryption on the Central's side.

Wu et al. [35] have analyzed all association models and their interplay in all stages in BR/EDR, BLE, and Bluetooth Mesh using ProVerif. They confirmed 5 known vulnerabilities and discovered two new attacks. The first attack is the Authentication Neutralization (BlueMAN) attack, in which the adversary forwards the provisioning information in Bluetooth Mesh and records the transcript to derive the session key and decrypt the data. The second attack is the Cross Stack Illegal Access (CSIA) attack, which, like the BLURtooth attack [4], exploits the cross-platform mechanism but requires one of the devices to be compromised.

Jangid, Zhang, and Lin [19] have analyzed the PKE association model using Tamarin. They confirmed known attacks, such as the Method Confusion Attack [32], the reflection attack [12], the static passkey reuse attack [29], and uncovered two new attacks: Group Guessing and Ghost Keystroke. While the first one is a type of the static reuse attack, when several connections are in place and devices use the same passkey for concurrent sessions, the second is a type of semi-honest connection, similar to the attack on PKE shown by Zhang et al. in [37]. They also suggested countermeasures for the already known and newly discovered attacks: ban on passkey reuse, check on the equality to the own values, distinguished UI for different association models, and device lock to avoid compromise.

Claverie et al. [11] have analyzed using Tamarin all the association models and protocols available in BR/EDR. They repeated the analysis separately for BLE and Bluetooth Mesh. They found an attack they call pairing confusion, in which the adversary downgrades to a Legacy protocol in one of the connections, what results into confusion with the SSP association models.

Shi et al. [27] have analyzed all association models and their interplay in all stages of the BLE SC protocol using Tamarin. They confirmed the Method Confusion Attack [32] and found a new confusion attack for the key size. To overcome these two attacks, the authors proposed countermeasures for PKE and NC: run the protocol twice and include the previously generated passkey/digits together with the data from the pairing request/response as an input into the second stage.

4.3 Suggested Countermeasures

Researchers suggested various fixes for Bluetooth protocols. Some fixes add modifications to existing protocols, some design completely new protocols. In the following, we describe the most relevant suggested fixes and discuss their advantages and drawbacks.

After the analysis of PKE, Troncoso and Hale [31] proposed two modifications to partly achieve CYBORG-security for this association model. The first modification (called Secure Hash) aims to authenticate various values: input and output capabilities should be included in the computation of the confirmation values; the nonces produced during the 20 iterations in PKE should be concatenated together with the initiator/responder role in the session, and the resulting concatenation is used in the computation of the confirmation values. The second modification (Dual Passkey Entry) suggests to use two independent passkeys that are generated by the both devices and transmitted to the other device via the user as a secure channel. This requires the devices to have keyboard-input capability (i.e., `KeyboardOnly` or `KeyboardDisplay` as *IOcap*), which limits the use of the subprotocol to the 4 cases only in total (out of 9 cases for PKE and 3 cases for NC in the current version of the protocol). It is noteworthy to mention that, according to the Bluetooth Core Specification, if both devices have `KeyboardDisplay` as *IOcap*, they agree on using the NC association model rather than PKE. These modifications still allow a number of the attacks and only mitigate the role confusion attack that the authors discovered.

To mitigate the Method Confusion, KNOB, and Keysize Confusion attacks, Shi et al. [27] suggested two fixes they claim to be backward compatible. Their patch implies including the parameters, negotiated during the pairing feature extraction, into the computation of the displayed passkey both in NC and PKE during the additional run of the Authentication stage 1. Besides the additional computation, this fix would imply creation of a new cryptographic function: It should take the previously generated passkey as an input, concatenate it with the both initiator and responder devices' parameters and truncate the output to 6 digits. The proposed fix does not help to guarantee the resistance to the KNOB and Keysize

Confusion attacks for JW, nor achieve authentication, since the downgrade to the JW protocol remains possible.

There are also non-cryptographic ways to address the attacks, e.g., safety number verification as in Signal or OOB communication. These are, however, of limited generality, as devices have different OOB capabilities, and require a technical add-on like an NFC chip, a camera, or a screen, which many Bluetooth devices do not have.

4.4 Related Cryptographic Frameworks

Jager et al. [17] suggested two generic compilers for BR-secure (security against passive adversaries is already sufficient) key exchange (KE) protocol, after which the derived key together with the transcript is handed over to the authentication protocol. The first compiler makes use of signatures and MACs, where the transcript from KE (and a random value) is signed using secret keys of parties and MAC is computed over the signatures using a MAC key derived from the resulting KE key. The second compiler reduces one additional round of exchanged messages by hashing the transcript, the MAC key, and a random value and signing the result with the secret key of the corresponding party. Their approach allows treating some protocols as pure KE protocols without authentication, which is exactly the case with Bluetooth.

Krawczyk [21] has introduced the notion of post-handshake authentication for TLS 1.3. This notion helps to capture the feature of TLS 1.3 with an optional authentication of the client that happens after the client and a server performed a handshake (key exchange in TLS) and the client already authenticated the server. Krawczyk suggested a compiler based on the signature and MAC solution (which can be generalized to other authentication mechanisms). The compiler functions as follows: the parties first derive a session key and a MAC key out of the obtained shared secret key (the option when a session key and a MAC key are concatenated and outputted as a shared secret is also possible but is not considered in the paper). Then the client signs the transcript exchanged during the handshake and sends it to the server together with the MAC-ed identities of the server and the client. The reason to have identities included in a separate MAC is to have a deniability notion.

TLS 1.3 also has a mode for usage of a pre-shared key (PSK), i.e., when two parties have a mechanism to establish a shared secret key, which can be used later for authenticating a TLS connection. This mode either requires the PSK to be shared out-of-band before, or the PSK must be derived from a previous *authenticated* connection, based on certified signatures in TLS 1.3. Link/long-term keys in Bluetooth, however, are usually established during an unauthenticated key exchange protocol, which is vulnerable to machine-in-the-middle attacks. Thus, such PSK methods in Bluetooth limit the adversary in the initial connection to being passive, which exactly corresponds to the TOFU-setting of [13]. Hence, deploying PSKs, which have been created in the pairing step, does not help in subsequent Bluetooth connections to protect against adversaries, which are active in the initial connection.

Schage et al. [26] have analyzed RFC for IKEv2 [20] as a privacy-preserving authenticated KE (PPAKE) protocol. That is, apart from achieving basic security guarantees, PPAKE ensures that the communication is deniable against the passive attacker, who can only eavesdrop the conversation. Their definition of the original key corresponds to our definition of TOFU connection. Although IKEv2 follows the similar 2-stage (KE-then-A) structure and allows two parties to derive somewhat long-term keys that might be used for deriving encryption keys later, it does not fully correspond to the way that Bluetooth devices are connecting: IKEv2 requires parties to store the transcript from the KE stage to sign it at the authentication stage. In addition, IKEv2 does not allow parties to create session keys before authentication, what should be allowed for Bluetooth solutions.

The solutions and compilers from [17] and [21] are not applicable to the Bluetooth setting, since the transcript of the pairing (KE) is not stored on the devices: After the successful initial pairing, only identities, L(T)K, and own DH share are available for reconnections and authentication. In addition, Bluetooth does not follow the regular structure of KE with the authentication happening right after the KE, but rather consists of the KE with the following reconnections.

5 Enhancing the TOFU Security Model

In this section, we give the security model for TOFU-or-DOFU-authenticated key exchange protocols. Besides the basic properties of key secrecy and match security in the TOFU-model [13], we also aim for authentication and weak forward secrecy. Since the authentication is optional and can be performed by any party at any point of time, we consider unilateral authentication, leaving out the execution of the authentication protocol once more with the reversed roles for mutual authentication. Mutual authentication follows from unilateral authentication in both directions, since signing or MAC-computation is done over fresh challenges from both sides, and the party applies the ConnectKey to at least own challenge (derived from the input that corresponds to the sid).

We also capture the presence of a passive adversary, who only eavesdropped on the conversation during the initial connection without intervening, by introducing TOFU-authentication. While it is not possible to achieve forward secrecy for Bluetooth with regard to the link/long-term key or reused DH shares, we still aim for the weaker version: that is, the session keys remain secure as long as the initial connection was not under active attacks, even if the authentication key of the party was corrupted. This corresponds to the cases, when, e.g., the creation process of the vendor was compromised.

For convenience, we follow the style of [13] to model different steps separately: we use separate sessions of the initial-connection step for link/long-term key agreement (with empty session keys); a reconnection step for encryption key derivation; and an authentication step. While the reconnection step normally follows the initial connection, the authentication step can be performed at any point of time, therefore, we let adversary decide on when to perform either of them.

5.1 TOFU-or-DOFU Security Model

Our TOFU-or-DOFU model follows the TOFU-model from [13] which, in turn, is based on the common game-based security model of [6]. The gist of the model is to use the flag isTOFU to indicate if the connection key (i.e., the link key in BR/EDR or the long-term key in BLE) has been established in an execution with an honest partner or not. If the flag is not set, then the model does

not consider this key to be a viable target, as it could be trivially known by the adversary anyway. We modify the TOFU-model to add authentication requirements, saying that even if the isTOFU flag is not set, but the authentication step has been carried out by the partner (flag isPartnerAuth is set), then the connection key is an admissible attack target. The modifications are highlighted with a different color.

Transmitted identities of the parties are defined as their Bluetooth addresses BD_ADDR and are usually denoted as A and B for the corresponding parties. The parties learn the identity of the intended partner before the KE execution through the corresponding discovery mechanisms. Note that the Bluetooth address does not match to the true identity (i.e., MAC-addresses) and can easily change in different sessions. To distinguish the transmitted address BD_ADDR from the actual device address, which we authenticate cryptographically, we denote the latter by MAC_ADDR. The true identity is globally unique and authenticated only later during the authentication step, and the Bluetooth address itself is not authenticated. We denote by $\mathcal{I}$ the universe of all possible identities MAC_ADDR.

Authentication Keys Linked to Identities. The Bluetooth protocols themselves are not equipped with public keys linked to the identities. Instead, some weak form of authentication should be provided through comparison or input of digits via PKEand NC, or out-of-band in OOB. Since such methods turn out to be highly vulnerable, and numerous attacks prove that they do not provide the common security guarantees, we instead revert to the classical authentication methods via certified long-term keys.

We assume each party, which intends to authenticate, possesses a long-term authentication key *authsk* and a matching verification key *authpk*. This can, for example, be a pair of signing and verification keys. The authentication keys are used for mutual or unilateral authentication. The certificate should be issued to the device's (static) identity i = MAC_ADDR, either the registered public address in BR/EDR or BLE, or the static random address in BLE. The latter requires some form of bookkeeping of the certification authority to ensure that static random addresses do not repeat. Since we focus on BR/EDR with our solution, one may assume that the identity is the public address.

The verification key comes with a certificate *cert* issued by some trustworthy entity. We denote the key pair of the certification authority as $(certsk, certpk) \leftarrow\!\!\$\ \mathsf{CertKGen}(1^\lambda)$, generated by algorithm CertKGen. We assume that all parties, including the adversary, have access to the certification authority's public key *certpk*. The certification of a public authentication key *authpk* is denoted by $cert \leftarrow\!\!\$\ \mathsf{Cert}(certsk, authpk)$.

While we allow parties to reauthenticate themselves (in case of credential change), we still consider this pair of keys as long-term secrets and allow the corruption of *authsk* via a corresponding Corrupt-oracle. This corruption does not model only inadvertent leakage of the secret key but also successful attacks on the public key. If oracle Corrupt is called with the input $i \in \mathcal{I}$, then the identity i is added to the (initially empty) set C of corrupt parties.

Further Keys. During the communication, parties derive a connection key ConnectKey for authentication during future reconnections. Although ConnectKey can be used for a long period of time, we treat it as an ephemeral key that can be derived anew via a corresponding InitSession-oracle. Parties also possess DH key pairs that they might use only once or in several initial connections. We allow parties to reuse Diffie–Hellman key pairs in several executions (as defined in the Bluetooth standard [8, Vol 2, Part H, Section 5.1]) by modeling initialization of keys[5] $(dhsk_i, dhpk_i) \leftarrow\!\!\$\ \mathsf{DHKGen}(1^\lambda)$ for each party i at the beginning of the game. The option to change the key pair used by party i is given to the adversary via the NextDH-oracle. This oracle models the key pair update via counter dhctr_i. The value of the counter is initially set to 0 and is incremented with each query to the oracle. Although these keys might be used in several connections, we do not allow the adversary to learn them (e.g., via a Corrupt query). This can reflect the behavior when a malicious app manages to get an access to the *L(T)K* but not to ephemeral keys like DH. We could capture the corruption of DH keys by tracking all the *L(T)Ks* and sessions keys derived from the corrupted DH share and forbidding the adversary from testing these sessions keys, but this sophisticates the model and the proof and does not help to yield forward secrecy of sessions keys.

Sessions. We call the k-th session run by the party with identity $i \in \mathcal{I}$ as a protocol session and mark it via an administrative label $\mathsf{lbl} = (i, k)$, $\mathsf{lbl} \in \mathcal{L}$, where $\mathcal{I}$ resp. $\mathcal{L}$ is the set of all identities resp. labels. Each session lbl consists of the following entries with boxed $\boxed{\text{initialized value}}$:

- id $\in \mathcal{I}$ defines the party's identity.
- pid $\in \mathcal{I}$ defines the identity of the intended partner. Note that this partner identity may only be authenticated later, when executing the authentication subprotocol.
- mode $\in$ {init, reconnect, auth} corresponds to the session in an initial-connection, reconnection, or authentication step.
- parent $\in \mathcal{L}$ for a reconnection or authentication session points to the initial session from which this session lbl is spawned off, or to itself if this is a parental session.
- state $\in$ {running, accepted, rejected} describes the state of the session: running once the session is initialized, and accepted or rejected if the party has accepted.
- aux stands for auxiliary information such as the association model (JW, PKE, NC, or OOB) in use; additional data transmitted out of band, e.g., passkey $\in \{0, 1, \ldots, 9\}^* \cup \{\perp\}$ in the PKE association model; or in authentication steps, the identity of the party aiming to authenticate (the "prover").
- dhctr stands for the counter of a Diffie–Hellman key pair that party i uses in the session. While the party always uses the same key pair according to the counter during one protocol session, in different sessions, the counter can be incremented and lead to the different key pairs used by the party.
- ConnectKey $\in \{0,1\}^* \cup \{\boxed{\perp}\}$) is the connection key that corresponds to the link key in Bluetooth BR/EDR and long-term key in Bluetooth LE. The connection key can be set during the initial connection. It is used to derive session keys during the reconnection and to provide key authentication during authentication.
- key $\in \{0,1\}^* \cup \{\boxed{\perp}\}$ is the session key that is set in reconnections (lbl.mode = reconnect) after the session accepts (lbl.state =

[5] We renamed the keys *dhsk*, *dhpk* from the terminology sk, pk used in [13] to distinguish them better from the other keys we have introduced here, like *authpk*, *certpk*.

accepted). For authentication (lbl.mode = auth) or initial connection (lbl.mode = init), the key is set to $\perp$ even after the session accepts.

- sid $\in \{0,1\}^* \cup \{\boxed{\perp}\}$ is the session identifier. The session identifier can be set only once when executing the protocol.
- isTested $\in$ {true, $\boxed{\text{false}}$} is a Boolean flag defining whether the session key has been tested before.
- isRevealed $\in$ {true, $\boxed{\text{false}}$} is the Boolean flag defining whether the session key has been revealed.
- isTOFU $\in$ {true, $\boxed{\text{false}}$} is a Boolean flag defining whether the session key has been derived in the honest execution during the initial connection.
- isPartnerAuth $\in$ {true, $\boxed{\text{false}}$} is a Boolean flag defining if the partner in entry pid has been authenticated through the authentication subprotocol. Note that this can only be set to true if the partner is still honest (pid $\notin C$).

Note that although Bluetooth connections are asymmetric in nature (i.e., Bluetooth uses initiator resp. responder roles, which transform into the Central resp. Peripheral roles—Bluetooth's analogue for client resp. server roles), we do not include roles into our model. The reason is that the choice of the roles is not fixed (while there are devices that can be only in the Peripheral role, dual-role devices also exist) and is not authenticated throughout the protocol. Cryptographically, it is neither relevant for the security property of the established authenticated key.

We follow the approach to model partnered sessions via the same session identifier.

Definition 5.1 (Partnered Sessions). We say two sessions lbl and lbl′ are partnered if lbl $\neq$ lbl′ and lbl.sid = lbl′.sid $\neq \perp$.

Adversary's Capabilities. We let an adversary $\mathcal{A}$ to be active and interact with the protocol through the following oracles:

InitSession(i, j, [aux]) creates a new session at party i with intended partner j and automatically incremented number k and returns lbl to the adversary. The identity of the party is assigned to the corresponding entry lbl.id $\leftarrow i$, the identity of the intended partner to lbl.pid $\leftarrow j$, the initial connection is set as the mode lbl.mode $\leftarrow$ init, and the state of the session changes to lbl.state $\leftarrow$ running. If optional auxiliary information [aux] is present, then it is stored in parameter lbl.aux, otherwise the parameter is set to $\perp$. The flags lbl.isTested,lbl.isRevealed, lbl.isPartnerAuth, lbl.isTOFU are set to their initial values false. The pointer to the parent lbl.parent is set to lbl. The session counter of a key pair receives the value of the party's current counter lbl.dhctr $\leftarrow$ dhctr$_i$.

Reconnect(lbl, [aux]) establishes a new session lbl′ = (i, k') from the parental session lbl and returns lbl′ if there exists a session with lbl.ConnectKey $\neq \perp$ and lbl.parent = lbl, otherwise returns $\perp$. A new session is established via calling InitSession(i, lbl.pid, [aux]) and overwriting lbl′.mode $\leftarrow$ reconnect as well as lbl′.parent $\leftarrow$ lbl. The new session lbl′ inherits the TOFU parameter lbl′.isTOFU $\leftarrow$ lbl.isTOFU, the authentication parameter lbl′.isPartnerAuth $\leftarrow$ lbl.isPartnerAuth, and the connection key lbl′.ConnectKey $\leftarrow$ lbl.ConnectKey from the parental session.

Authenticate(lbl, [aux]) establishes a new session lbl′ = (i, k') from parental session lbl and returns lbl′ if there exists the session with lbl.ConnectKey $\neq \perp$ and lbl.parent = lbl.parent, otherwise returns $\perp$. This oracle is used to establish a session for unilateral authentication. The choice is determined by placing the identity ($\{i\}$ or $\{j\}$) of the authenticating party (the "prover") into aux. Establishing a new session is done through calling InitSession(i, lbl.pid, [aux]) and overwriting lbl′.mode $\leftarrow$ auth as well as lbl′.parent $\leftarrow$ lbl. The new session lbl′ inherits the TOFU parameter lbl′.isTOFU $\leftarrow$ lbl.isTOFU and the connection key lbl′.ConnectKey $\leftarrow$ lbl.ConnectKey that it is about to authenticate from the parental session. We allow parties to reauthenticate themselves, therefore this oracle can be queried multiple times, even for the same lbl or same identity.

Send(lbl, m) sends a protocol message m to the session lbl and returns the answer. If the session does not exist or is not established, the oracle returns $\perp$. During the oracle execution, the session may change the state lbl.state to accepted or rejected and set session identifier lbl.sid. If the session accepts (lbl.state = accepted), then the following happens:

- If lbl.mode = init and there exists a session lbl′ partnered to lbl, then set lbl.isTOFU $\leftarrow$ true and lbl′.isTOFU $\leftarrow$ true.
- If lbl.mode = auth and the partner is authenticated (lbl.pid $\in$ lbl.aux) and is currently not corrupt (lbl.pid $\notin C$), then the partner authentication flag is set lbl.isPartnerAuth $\leftarrow$ true. Only in this case, authenticate all other related sessions by setting lbl′.isPartnerAuth $\leftarrow$ true for all sessions lbl′ with lbl′.parent = lbl.parent as authentication spans over the connection key. Note that this also sets the authentication flag for the parental session to true.
- If there exists a partnered session lbl′ with lbl′.isTested = true, the flag of the session here also receives lbl.isTested $\leftarrow$ true.
- If there exists a partnered session lbl′ with lbl′.isRevealed = true, then the flag of the session here also receives lbl.isRevealed $\leftarrow$ true.

NextDH(i) updates the Diffie–Hellman key pair used by party i and returns the public key part to the adversary. To update, the oracle increments counter dhctr$_i$ and generates a new key pair ($dhsk_i$[dhctr$_i$], $dhpk_i$[dhctr$_i$]) $\leftarrow\!\!\$$ DHKGen(1^λ). The new pair will be used only in future sessions, not in currently running.

Reveal(lbl) returns the session key key of session lbl. If the session does not exist or if lbl.state $\neq$ accepted, the oracle returns $\perp$. Else sets lbl.isRevealed $\leftarrow$ true and for all partnered sessions lbl′ with lbl′.sid = lbl.sid also sets lbl′.isRevealed $\leftarrow$ true.

Corrupt(i) returns the secret authentication key *authsk* of party $i \in \mathcal{I}$. If the party does not have an authentication key, the oracle returns $\perp$, otherwise sets $C \leftarrow C \cup \{i\}$. Note that although the connection keys might be used for several sessions, we do not allow their corruption, but only of the secret authentication key.

Test(lbl) is the oracle for testing the session key key of session lbl. If the session does not exist, or has not accepted (lbl.state $\neq$ accepted), or has been already revealed (lbl.isRevealed = true) or tested (lbl.isTested = true), or is neither the result of the genuine pairing (lbl.isTOFU = false) nor authenticated the partner yet (lbl.isPartnerAuth = false), or key = $\perp$, then the query returns $\perp$. Otherwise, for the challenge bit b, the query returns either

the real key key if b = 1, or a random string of length |key| if b = 0. To prevent the adversary from an easy win by querying this oracle to the same or partnered session again, the session lbl sets the flag lbl.isTested ← true and all partnered sessions lbl′ with lbl′.sid = lbl.sid set the flag lbl′.isTested ← true.

Note that according to our model, the authentication step is not executed over the secured communication channel, protected under an encryption key (as potentially Bluetooth would do), but run "in plain". We can model encrypted authentication steps by having the parties execute a reconnection step first (to establish an encryption key), immediately revealing this encryption key, followed by an authentication step in which the revealed encryption key can be used to incorporate encryption. Since our solutions would still be secure if the authentication steps were not encrypted, we stick to the easier model here.

5.2 Match Security and Key Secrecy

We define the two desired properties of authenticated key exchange protocols: match security and (session) key secrecy. The first property stands for the attacks where the adversary manages to confuse honest parties about their intended partners or with whom they share the session key. Key secrecy guarantees that the key is hidden from the adversary. For all security properties, including also the authentication property later, we assume the same initialization procedure, which generates the users' keys and certifies them. This is captured by the following description:

Initialize(λ)

$b \leftarrow\$ \{0,1\}$, $C \leftarrow \emptyset$
$(certsk, certpk) \leftarrow\$ \mathsf{CertKGen}(1^\lambda)$
forall $i \in \mathcal{I}$ ***do***
  $\mathsf{dhctr}_i \leftarrow 0$
  $(dhsk_i[0], dhpk_i[0]) \leftarrow\$ \mathsf{DHKGen}(1^\lambda)$
  $(authsk_i, authpk_i) \leftarrow\$ \mathsf{AuthKGen}(1^\lambda)$
  $cert_i \leftarrow\$ \mathsf{Cert}(certsk, authpk_i)$

All values are available in the subsequent games.

Match Security. Match security ensures that two partnered sessions hold the same session key (1), and at most two sessions are partnered (2). Since the same ConnectKey can be used in several reconnections, we also need to require the ConnectKey array used for reconnection or authentication to match the one used in the initial connection. This is reflected in the split of the first condition into two cases: for initial connections, where we require matches on the session key and the connection keys, and for reconnections/authentication, where we require session key matching under the condition that the sessions are partnered and start with the same connection key array.

Definition 5.2 (TOFU-or-DOFU Match Security). A key exchange protocol Π provides TOFU-or-DOFU Match Security if for any PPT adversary $\mathcal{A}$ and identity set $\mathcal{I}$

$$\mathbf{Adv}^{\text{Match Security}}_{\mathcal{A},\Pi,\mathcal{I}}(\lambda) := \Pr\left[\mathbf{Exp}^{\text{Match Security}}_{\mathcal{A},\Pi,\mathcal{I}}(\lambda) = 1\right]$$

is negligible in the following experiment:

$\mathbf{Exp}^{\text{Match Security}}_{\mathcal{A},\Pi,\mathcal{I}}(\lambda)$

Initialize(λ)
$\mathcal{A}^{\mathsf{O}}(certpk, \{(i, dhpk_i[0], authpk_i, cert_i)\}_{i\in\mathcal{I}})$
return 1 ***if*** $\exists$ pairwise distinct lbl, lbl′, lbl″ $\in \mathcal{L}$:
(1*a*) lbl.sid = lbl′.sid $\neq \perp$ **and** lbl.mode = init **and**
  (lbl.key $\neq$ lbl′.key **or** lbl.ConnectKey $\neq$ lbl′.ConnectKey),
(1*b*) lbl.sid = lbl′.sid $\neq \perp$ **and** (lbl.mode = reconnect **or** auth) **and**
  lbl.ConnectKey = lbl′.ConnectKey **and** lbl.key $\neq$ lbl′.key,
(2) lbl.sid = lbl′.sid = lbl″.sid $\neq \perp$

where O comprises all oracles InitSession, Reconnect, Authenticate, Send, NextDH, Reveal, Corrupt, Test described in Section 5.1.

Key Secrecy. This property ensures the session key remains secret if an adversary mounts an attack. Note that we capture the secrecy of the trustworthy or authenticated sessions only, what is ensured by the attack model (i.e., adversary is forbidden from querying Test-oracle to sessions with isTOFU = false and isPartnerAuth = false).

Definition 5.3 (TOFU-or-DOFU Key Secrecy). A key exchange protocol Π provides TOFU-or-DOFU Key Secrecy if for any PPT adversary $\mathcal{A}$ and identity set $\mathcal{I}$

$$\mathbf{Adv}^{\text{Key Secrecy}}_{\mathcal{A},\Pi,\mathcal{I}}(\lambda) := \Pr\left[\mathbf{Exp}^{\text{Key Secrecy}}_{\mathcal{A},\Pi,\mathcal{I}}(\lambda) = 1\right] - \frac{1}{2}$$

is negligible in the following experiment:

$\mathbf{Exp}^{\text{Key Secrecy}}_{\mathcal{A},\Pi,\mathcal{I}}(\lambda)$

Initialize(λ)
$b' \leftarrow\$ \mathcal{A}^{\mathsf{O}}(certpk, \{(i, dhpk_i[0], authpk_i, cert_i)\}_{i\in\mathcal{I}})$
return 1 ***if***
  $b' = b$ **and** there are no lbl, lbl′ $\in \mathcal{L}$ with lbl.sid = lbl′.sid
    but lbl.isRevealed = false and lbl′.isTested = true

where O comprises all oracles InitSession, Reconnect, Authenticate, Send, NextDH, Reveal, Corrupt, Test described in Section 5.1.

6 Authentication

We discuss the full security model for authentication protocols in the full version of the paper [14]. It follows a similar approach as the security of key exchange models, allowing the adversary to interact with authentication sessions, and we use the same entries for sessions as in the key exchange case. In particular, sessions lbl will set session identifiers sid upon acceptance and set the flag isPartnerAuth if the intended (honest) partner has authenticated successfully. We require three security properties of good authentication protocols Ψ, all defined formally in the full version [14]:
Match Security: Similarly to the key exchange setting, we require that no three honest sessions derive the same session identifier sid and are thus considered to be partnered.
Authentication: This means that if an accepting session lbl considers the intended partner to be authentic (lbl.isPartnerAuth = true), then there is indeed an honest session, which is partnered (and has thus executed this session) and which also holds the same connection key lbl.ConnectKey.

Leakage Resilience: Unless the adversary trivially knows the connection key, e.g., if it participated on behalf of a corrupt party, the authentication protocol hides ConnectKey in the sense that using an independent random key instead of ConnectKey is indistinguishable.

We describe here four main variants of leakage-resilient authentication protocols, two more suited to BR/EDR and two more suited to BLE (albeit we cannot prove security of the overall key exchange protocol with deferrable authentication for BLE, these protocols still provide secure authentication). In all schemes, we let the authenticating party include the *IO capabilities request* (BR/EDR) resp. *pairing request* data, describing the device's supported features. These data are transmitted before the secure simple pairing and include the device's IO capabilities, the OOB data flag, the authentication requirements (for BR/EDR), as well as the maximum encryption key size, and the initiator and responder key distribution entries (for BLE). For BR/EDR, this value *IOcap* can be described with 24 bits (as in key derivation) and the value *PairReq* in BLE with 48 bits. We let the authenticating device send this as part of the authentication protocol. Note that the KNOB attack on BLE [3], for instance, uses the fact that the adversary is able to modify the maximum encryption key size when this value is transmitted in the initial pairing. Including this value here as part of *PairReq* in the authentication step implies that this would be detected later during authentication.

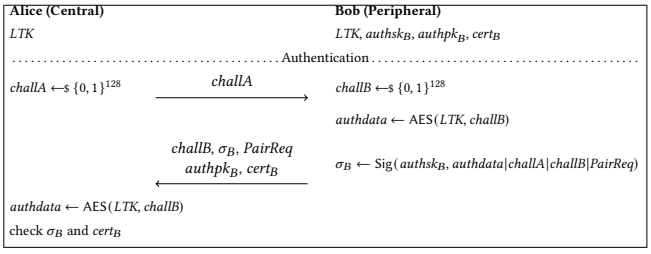

Figure 2: Signature-based Bluetooth Authentication Subprotocol for BLE. Recall that *PairReq* describes the device's preference transmitted also in the pairing request or response step. The session identifier here and for alternative authentication protocols is given as sid = (***challA***, ***challB***)**. The session key** KDF **in all cases is** $\perp$**.**

Our first protocol uses (certified) signatures to authenticate, where the signed data is computed in the same way as an encryption key from the long-term key, AES(*LTK*, *challB*), where *challA* and *challB* are random 128-bit values, which are appended to the AES value to form *authdata*. The prover sends a signature σ_B of *authdata* under its signature key *authsk* and appends the certificate for the verification key (which includes the verification key *authpk*). In this first variant, we rely on the key-collision resistance of AES in the sense that finding another key LTK' mapping *challB* to the same value should be infeasible. The exact requirement appears in the full version of the paper [14]. The advantage is that the computation of the signed data complies with encryption key derivation in BLE with security function *e* in [8]. The security proofs appear in the full version of the paper [14].

We define here for simplicity the set of admissible test values $\mathcal{X}^{\text{auth}}_{\text{test}}$ to be $\{0,1\}^{128}$. Strictly speaking, since $\mathcal{X}^{\text{auth}}_{\text{test}}$ already covers all possible input values for AES($LTK, \cdot$), the adversary could not call AuthSide about non-trivial values anymore. We could be more liberal and define $\mathcal{X}^{\text{auth}}_{\text{test}}$ to be the set of random values *challB* appearing in test queries only and allowing for other queries about AES(LTK, x), but then we would need to adapt the model slightly to set $\mathcal{X}^{\text{auth}}_{\text{test}}$ randomly. We omit this extension here.

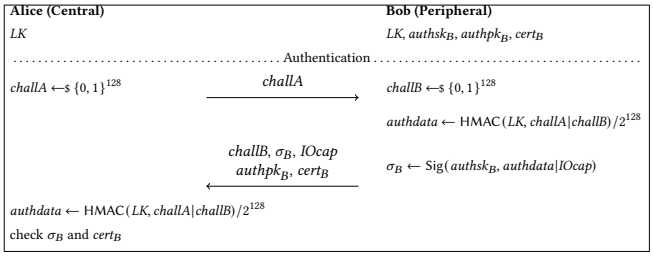

Figure 3: Signature-based Bluetooth Authentication Subprotocol for BR/EDR. Recall that *IOcap* describes the device's IO capabilities (as used during secure simple pairing). Then notation HMAC(***LK***, ***challA***|***challB***)$/2^{128}$ **means to take the leftmost** 128 **bits of the hash function's output.**

Alternatively, and this constitutes our second variant, we may use a collision-resistant hash function Hash directly and, instead, compute *authdata* = HMAC(*LK*, *challA*||*challB*) and sign these data as before, this time with the *IOcap* features. The advantages here are that the security is based on the common collision resistance of the underlying hash function Hash in truncated HMAC and that this approach can easily support longer challenge values, like the concatenation of two 128-bit challenges. In fact, the HMAC computation resembles the device authentication confirmation function $h5$ in BR/EDR.[6] We set $\mathcal{X}^{\text{auth}}_{\text{test}}$ to be the set of all strings except for 128- and 192-bit values; note that BR/EDR calls HMAC($LK, \cdot$) in reconnection steps only about these two input lengths.

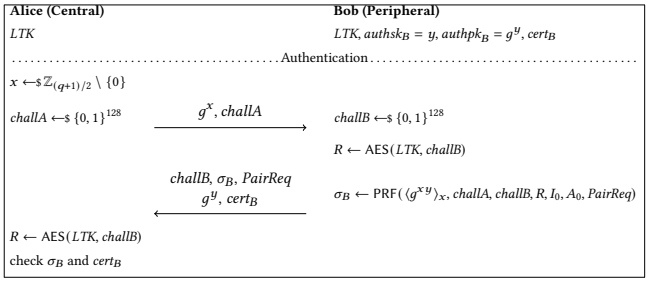

Figure 4: DH-based Bluetooth Authentication Subprotocol for BLE (with PRF **being based on** AES-CMAC**), where** $I_0 = 0^{|\textbf{\textit{IOcap}}|}$ **and** $A_0 = 0^{|A|}$ **are placeholders, and *PairReq* describes the** 48 **bits of the pairing request data.**

The third variant, for BLE, avoids signatures and is thus more deniability-friendly, i.e., the participation in the protocol cannot

[6] Strictly speaking, function $h5$ uses the device key as input, derived from the link key via function $h4$; this extra step may also be done here.

be traced. To this end, we assume that the prover holds a certified Diffie–Hellman share g^y and the verifier provides a random Diffie–Hellman share g^x. Both parties also exchange 128-bit nonces *challA*, *challB* and compute a confirmation value as $\sigma_B \leftarrow \mathsf{PRF}(\langle g^{xy}\rangle_x, challA, challB, R, I_0, A_0, PairReq)$ where $I_0 = 0^{|IOcap|}$ and $A_0 = 0^{|A|}$ are the placeholders for the prover's IO capabilities and the parties identities, *PairReq* are the 48-bit pairing request data, and R is given by $\mathsf{AES}(LTK, challB)$. The reason for computing σ_B this way is that it corresponds to the confirmation value computation in the pairing step (LE Secure Connections check value generation function $f6$ for BLE in [8]), and is based on AES-CMAC for BLE, with extra steps to map the Diffie–Hellman value to a 128-bit key (LE Secure Connections key generation function $f5$ in [8]). Since the values I_0, A_0 do not serve any purpose here, beyond this compatibility issue, and may even not be available at this point, we set them to 0. The prover finally sends σ_B, the certificate, and the challenge value to the verifier. Once more let $\mathcal{X}^{\text{auth}}_{\text{test}} = \{0,1\}^{128}$.

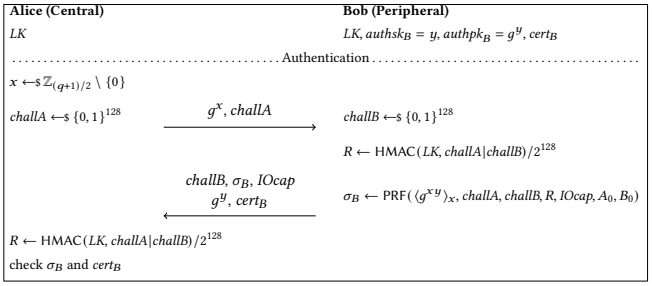

Figure 5: DH-based Bluetooth Authentication Subprotocol for BR/EDR (with PRF being being a truncated HMAC), where *IOcap* are the device's IO capabilities, and $A_0 = 0^{|A|}$, $B_0 = 0^{|B|}$ are placeholders.

The fourth variant maps the third scheme to BR/EDR, once more using Diffie–Hellman. The only difference to the previous version is that HMAC (truncated to 128 bits) is used to compute the value R and that we place the 24-bit *IOcap* describing the device's capabilities into the corresponding entry instead of I_0 and now also set the second address entry $B_0 = 0^{|B|}$. Let $\mathcal{X}^{\text{auth}}_{\text{test}} = \{0,1\}^{256}$.

7 Security of the BR/EDR Protocol with Deferrable Authentication

We next argue that the security of the BR/EDR Bluetooth protocol can be enhanced via deferrable authentication, lifting the guarantees from trust-on-first-use to trust-on-first-use *or* authenticated. This means that any session key, future or past, of an authenticated connection is also secret, even if the adversary has previously been active during the initial pairing.

The proof strategy is intuitive: The difference from the TOFU model is that the adversary may also test sessions where it was active during the initial interaction, as long as there has been a subsequent authentication step. Yet, the security of the authentication protocol ensures that only the intended honest partner could have executed this part and also holds the correct connection key. The latter, however, implies that this party must have indeed been the one that executed the initial connection and computed the connection key. Hence, the authentication step basically confines the adversary once more to a TOFU attack. Leakage resistance of the authentication protocol ensures that the extra deployment of the connection key in the authentication step does not facilitate the adversary's TOFU-only attack.

The proofs of Match Security and Key Secrecy for our protocols appear in the full version of the paper [14].

7.1 Match Security

Before discussing the key secrecy property, we first look at match security. We define the session identifier sid of our protocol as follows. For the initial connections, sid consists of the x-coordinates of the DH-shares, Bluetooth addresses, and randomly generated nonces: $\text{sid} = (\text{init}, \langle g^a\rangle_x, \langle g^b\rangle_x, A, B, Na, Nb)$. For reconnections in BR/EDR, sid should contain the randomly generated nonces as well as the Bluetooth addresses of the devices: $\text{sid} = (\text{reconnect}, AU\_RAND\_C, AU\_RAND\_P, A, B)$. Finally, for authentication, we inherit the session identifier sid' of the authentication protocol but prepend the key word auth, $\text{sid} = (\text{auth}, \text{sid}')$.

Proposition 7.1 (BR/EDR Match Security). *The BR/EDR Bluetooth protocol Π with an authentication protocol Ψ and a certification scheme Γ provides TOFU-or-DOFU-Match Security. That is, for any adversary $\mathcal{A}$ initiating at most q_s sessions, there exists an adversary $\mathcal{B}$ (with approximately the same running time as $\mathcal{A}$ and initiating also at most q_s sessions), such that*

$$\boldsymbol{Adv}^{\textit{Match Security}}_{\mathcal{A},\Pi,\mathcal{I}}(\lambda) \le q_s^2 \cdot 2^{-|nonce|} + \boldsymbol{Adv}^{\textit{Match Security}}_{\mathcal{A},\Psi,\Gamma,\mathcal{I}}(\lambda),$$

where $|nonce| = 128$.

The bound follows from the birthday bound for colliding nonces *Na*, *Nb* in initial connections resp. *AU_RAND_C*, *AU_RAND_P* in reconnections, all of 128 bits. In addition, we inherit the Match Security bound for authentication steps. The full proof appears in the full version of the paper [14].

7.2 Key Secrecy

As explained earlier in this section, the proof strategy is to reduce the adversary to a TOFU-only adversary. For technical reasons, yet, we still need to dive into the original TOFU proof in [13]. The underlying cryptographic assumptions (like the PRF-ODH assumption or collision resistance of truncated HMAC) are stated formally in the full version of the paper [14].

Proposition 7.2 (BR/EDR Key Secrecy). *The BR/EDR Bluetooth protocol Π with an authentication protocol Ψ (with set $\mathcal{X}^{auth}_{test} = \{0,1\}^* \setminus (\{0,1\}^{128} \cup \{0,1\}^{192})$) and a certification scheme Γ provides TOFU-or-DOFU-Key Secrecy: for any adversary $\mathcal{A}$ initiating at most $q_s = q_{init} + q_{rec} + q_{auth}$ sessions (of which q_{init} are initial connections, q_{rec} are reconnections, and q_{auth} are authentication sessions), there exist efficient adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}, \mathcal{E}$, and $\mathcal{F}$ (with roughly the same running time as $\mathcal{A}$), such that*

$$\begin{aligned}\boldsymbol{Adv}^{\textit{Key Secrecy}}_{\mathcal{A},\Pi,\mathcal{I}}(\lambda) \leq\ & q^2_{init}\cdot 2^{-|nonce|} + \boldsymbol{Adv}^{\textit{Authentication}}_{\mathcal{B},\Psi,\Gamma,\mathcal{I},\mathcal{X}^{auth}_{test},\mathsf{HMAC}/128}(\lambda)\\ &+ \boldsymbol{Adv}^{CR}_{C,\mathsf{HMAC}/128}(\lambda) + q^3_{init}\cdot \boldsymbol{Adv}^{\textit{PRF-ODH}}_{\mathcal{D},\mathsf{PRF},\mathbb{G}}(\lambda)\\ &+ 2\cdot \boldsymbol{Adv}^{\textit{Leakage Resistance}}_{\mathcal{E},\Psi,\Gamma,\mathcal{I}}(\lambda)\\ &+ 2q_{rec}\cdot \boldsymbol{Adv}^{PRF}_{\mathcal{F},\mathsf{HMAC}/128}(\lambda) + q^2_{rec}\cdot 2^{-|ACO|},\end{aligned}$$

where $|nonce| = 128$ *and* $|ACO| = 64$.

Note that the bound roughly matches the one in [13] but adds the (un)forgeability bound for the authentication step, as well as the leakage bound for the link key in the authentication step. Indeed, the factor q^3_{init} in the PRF-ODH advantage and the factor $q^2_{rec}\cdot 2^{-|\mathrm{ACO}|}$ mean that the bounds are only reasonable if one considers corresponding bounds on the number of sessions, which can occur in a setting.

Let us highlight the term $\mathbf{Adv}^{\mathrm{CR}}_{C,\mathrm{HMAC}/128}(\lambda)$ in the above bound. This term captures the likelihood of collisions when deriving the link key via truncated HMAC in BR/EDR and is used in the proof to argue that the authentication step (which ranges also over the link key) must be carried out by the same party as in the initial connection (in which exactly this link key has been created). Our results would transfer to BLE almost immediately, if we could argue a similar term for the collision resistance of the long-term key derivation function AES-CMAC in BLE. Unfortunately, this would be hard to show, especially in light of known malleability attacks [12].

The proof appears in the full version of the paper [14]. It follows the strategy for the TOFU-case from [13] closely. Specifically, consider an attack against a session, which has either been involved in a TOFU-execution with an honest party—in which case, key secrecy follows as before—or has successfully executed the authentication step with the other party instead (DOFU case). We only need to ensure key secrecy for such sessions. In an additional game hop, we argue that executing the authentication step can only be done by the honest intended partner (adding the term $\mathbf{Adv}^{\mathrm{Authentication}}_{\mathcal{B},\Psi,\Gamma,\mathcal{I},\mathcal{X}^{\mathrm{auth}}_{\mathrm{test}},\mathrm{HMAC}/128}(\lambda)$). Furthermore, by the collision resistance of the key derivation in BR/EDR, captured by adding the term $\mathbf{Adv}^{\mathrm{CR}}_{C,\mathrm{HMAC}/128}(\lambda)$ to the bound, there must be a unique session in which this honest party connected to the session under attack. However, this lands us again in the TOFU-case. Taking into account the negligible leakage of the long-term key used in authentication, captured by $2\cdot\mathbf{Adv}^{\mathrm{Leakage\ Resistance}}_{\mathcal{E},\Psi,\Gamma,\mathcal{I},\mathcal{X}^{\mathrm{auth}}_{\mathrm{test}},\mathrm{HMAC}/128}(\lambda)$ in the bound, key secrecy in the TOFU-or-DOFU-setting thus follows.

8 Known Attacks on Bluetooth and our Mitigation

In this section, we briefly describe in how far our solution prevents known attacks. We start with an example of the Ghost Keystroke attack on Bluetooth from [37] and show how it is mitigated by our solution. We continue with a short summary of other known attacks on Bluetooth and explain the efficacy of our method against these attacks. More detailed explanation of the attacks and their fixes can be found in in the full version of the paper [14].

8.1 Example of the Ghost Keystroke Attack

The source of the Ghost Keystroke Attack is the impersonation attack on PKE discovered by Zhang et al. [37] (see Figure 6 for a pictorial description). In this attack, a user tries to connect two devices, one of which (Central A) has a screen to display the digital passkey, and the second one (Peripheral B) has an input capability for the user to enter the digits. If an MitM-attacker M had prior access to the input device B and established a connection (by sending its own DH public share g^m and computing the compromised LK_{comp}), it can receive the keystroke entered at this compromised device. When the user initiates a connection between devices A and B, the attacker connects to the output device A, which displays the digits. The user enters those digits on the compromised input device B, which sends the keystroke with the passkey to the attacker. Eventually, the attacker successfully impersonates the input device B and connects its fake input device to the user's honest output device A without the user noticing.

The impersonation attack on PKE from [37] was generalized by Jangid et al. [19] as the Ghost Keystroke Attack. The attack assumes that one device is leaking the passkey: e.g., if the adversary had a physical access to the device and established the connection during the user's absence, then the attacker can receive the keystrokes via this connection whenever the user enters anything on this device. The Ghost Keystroke Attack is valid for PKE, and even the fix for PKE suggested in [27] and the Secure Hash Modification fix in [31] do not protect against it (the principle of the attack is the same as shown in Figure 6). When the adversary not only can receive the keystrokes but also inject malicious strings on the compromised device's screen, the attack is even extendable to Dual Passkey Entry in [31], unmodified NC, and the fix for NC in [27].

The adversary can launch the Ghost Attack on our scheme only if it has the power to forge in the authentication process, assuming it has been carried out at some point. Since this step also authenticates the link key LK, and the honest device derives different from the compromised device's LK in the corresponding sessions, the adversary will not be able to successfully authenticate, unless it breaks unforgeability of the authentication procedure.

8.2 Efficacy of Authentication Against Attacks

Prevented Attacks. The attacks we prevent are commonly addressing the capabilities exchange stage, where the adversary can get in between the devices and replace the genuine capabilities with the needed for itself without any further authentication. For example, the adversary can replace the IO capabilities in the exchanged messages and trick devices into the DH-like JW association model, as shown in several works, e.g., [15, 16]. If devices have screens, the adversary can launch even more sophisticated attacks, such as a method confusion attack [32] and its extension, pairing confusion attack [11]. With these attacks, the devices are tricked into using different association models, which look the same from the user perspective, and in case of the pairing confusion—even into different protocols that look alike. Changing initiator/responder packets [31] allows the adversary to trick the parties into the believe that each is the initiator/responder. While the attack itself does not lead to the key leakage, cryptographically such attacks should not be allowed in any good protocol.

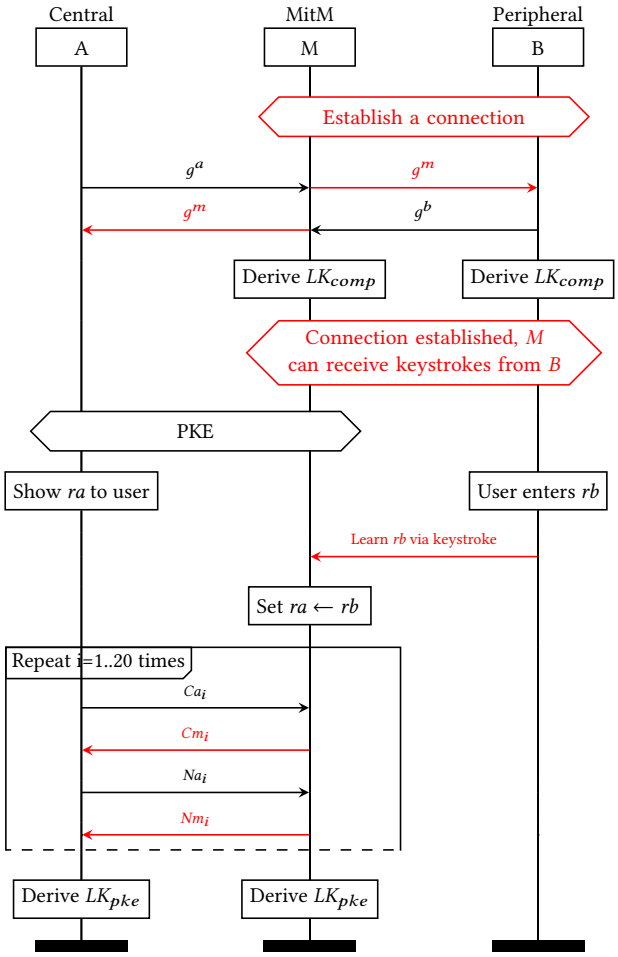

Figure 6: Message Flow for Ghost Keystroke Attack [19, 37] on Secure Hash Modification [31]. The MitM adversary *M* manages to impersonate *B* by establishing a connection with it to receive the leakage of the passkey**. Each connection is independent, and the honest devices derive different long-term keys in each: LK_{pke} by Central *A* resp. initially compromised LK_{comp} by Peripheral *B*.**

In each of the above attacks, the devices will derive pairwise-different connection keys. Hence, we can expect the authentication step to bring forward this discrepancy, as the keys *LK*s and hence the genuine signatures will not match—unless the adversary breaks authenticity, e.g., forges a signature.

We note that to prevent capability mismatches, both fixes in [31] and [27] suggest modifications to the concrete standalone protocols. They do not take into account the interplay of all the different association models. That is, the adversary can still downgrade the communication to the JW association model by simply modifying the *IOcap* of devices, which, in turn, will not even enter the modified protocol and simply stay at the less secure association model.

Potentially Prevented Attacks. The prevention of some attacks is not possible because of some vulnerable parts of the Bluetooth protocol. For instance, the protocols in BLE employ the weak function AES-CMAC that is not collision resistant. Bluetooth has a cross-platform mechanism to derive a key on one transport channel using the key derived from another transport. The cross-key derivation mechanism, however, uses AES-CMAC for these procedures: it would not be possible to carry out our proof for this function, due to similar weaknesses as in the standalone BLE setting. The KNOB attack [3] and the Keysize Confusion attack [27] in BLE could have been mitigated in principle, as the negotiation is done during the capability exchange stage. Yet, due to the usage of the non-collision-resistant function AES-CMAC, we cannot show security against these attacks in BLE.

Attacks Escaping our Solution. In BR/EDR, attacks on the negotiation mechanism [3, 5] cannot be prevented by our method, as the negotiation mechanism has its own protocol, independent from Bluetooth pairing. In addition, all Legacy protocols do not withstand against the passive adversary (as shown, e.g., in [18, 25]) and hence are not possible to be secured. Initiator resp. responder roles are tied to the Central resp. Peripheral roles in Bluetooth; by exploiting the challenge-response procedure in Legacy reconnections in BR/EDR [2], the adversary can make only the Central to have asked the Peripheral to respond to the challenge itself. If the Central device is a target, the attacker asks for a role switch and exploits the one-wayness again. For Secure Connections, the attacker can downgrade to Legacy reconnections. The attacks themselves do not leak or help the adversary to learn the link key *LK*, nor the encryption keys. The current version of the Core Specification forbids the downgrade to Legacy reconnections for Secure Connections pairing, disallows role switching during the reconnection in progress, and enforces the mutual authentication in Legacy reconnections.

Other attacks. The reflection attack [12] allowed the adversary to use a connection with one party to learn the secret passkey by simply reflecting the values sent by this party. This passkey was then used in a connection with another party to impersonate the first. This attack was mitigated in the Bluetooth Core Specification v.5.3, which mandates to check the equality of the received values to their own sent values.

A similar idea to KNOB and BIAS is used in BLUFFS attacks [1]. The attack exploits the Legacy encryption key derivation, which bases the randomness of the key on only one nonce from a Central. Hence, the adversary can force the Peripheral to reuse the same nonce in multiple sessions. This is only fixable by forbidding the usage of the same nonce.

An implementation drawback with the wrong handling of the errors [33] allowed accessing the attributes, and spoofing the responses [33] could lead to the "impersonation" of one device towards another. The latter attack may only be fixed by enforcing the encryption by default, as the attack does not happen during the key establishment or authentication stages but rather after the encryption should have been enabled.

Two relay attacks (two-sided and one-sided) [22] targeted the Legacy reconnection in BR/EDR. The first attack simply relays the messages in the Legacy mutual reconnection step between two parties, the second allows relaying the challenge from one party to another by terminating one of the connections. These attacks do not lead to the adversary learning the key, hence we leave the attacks out of our scope.

9 Conclusion

In this work, we proposed a scheme based on certificates to add authentication to the Bluetooth Secure Connections protocol and to prevent many known attacks on the protocol. We showed the cryptographic strength of the solutions in an extension of common game-based security models. It is noteworthy that our solution only yields a provably secure version for BR/EDR, wherein HMAC is used to derive link keys. In contrast, our proof technique falls short of working in the BLE setting, where the long-term key is computed via AES-CMAC. The reason lies in the lack of collision resistance of the "hashing function AES-CMAC" [8]. It would be easy to lift our proof if, say, also HMAC would be used in BLE. Indeed, we are positive that one could then also extend our result to the cross-transport key derivation (CTKD) mechanism for dual-mode devices, wherewith a BR/EDR link key can be converted to a BLE long-term key and vice versa. This would allow us to also rule out corresponding CTKD attacks. Given the security issues with AES-CMAC, also the malleability problems in Bluetooth Mesh with this function [12], it may be worth to consider switching.

Acknowledgments

We thank the anonymous reviewers and the shepherd for the valuable feedback that helped to improve the paper. This research work has been funded by the German Federal Ministry of Education and Research and the Hessian Ministry of Higher Education, Research, Science and the Arts within their joint support of the National Research Center for Applied Cybersecurity ATHENE. It has also been funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) — 251805230/GRK 2050 and 23661529/SFB 1119.

References

[1] D. Antonioli. BLUFFS: Bluetooth forward and future secrecy attacks and defenses. In *ACM CCS 2023*, pages 636–650, 2023.
[2] D. Antonioli, N. O. Tippenhauer, and K. Rasmussen. BIAS: Bluetooth impersonation AttackS. In *2020 IEEE Symposium on Security and Privacy*, pages 549–562, 2020.
[3] D. Antonioli, N. O. Tippenhauer, and K. Rasmussen. Key negotiation downgrade attacks on bluetooth and bluetooth low energy. *ACM Trans. Priv. Secur.*, 23(3), 2020.
[4] D. Antonioli, N. O. Tippenhauer, K. Rasmussen, and M. Payer. BLURtooth: Exploiting cross-transport key derivation in bluetooth classic and bluetooth low energy. In *ASIACCS 22*, pages 196–207, 2022.
[5] D. Antonioli, N. O. Tippenhauer, and K. B. Rasmussen. The KNOB is broken: Exploiting low entropy in the encryption key negotiation of bluetooth BR/EDR. In *USENIX Security 2019*, pages 1047–1061, 2019.
[6] M. Bellare and P. Rogaway. Entity authentication and key distribution. In *CRYPTO'93*, pages 232–249, 1994.
[7] E. Biham and L. Neumann. Breaking the bluetooth pairing - the fixed coordinate invalid curve attack. In *SAC 2019*, pages 250–273, 2019.
[8] Bluetooth Special Interest Group (SIG). Bluetooth core specification, 2023. Ver. 5.4.
[9] M. Cäsar, T. Pawelke, J. Steffan, and G. Terhorst. A survey on bluetooth low energy security and privacy. *Comput. Netw.*, 205(C), 2022.
[10] R. Chang and V. Shmatikov. Formal analysis of authentication in bluetooth device pairing. In *Proceedings of LICS/ICALP Workshop on Foundations of Computer Security and Automated Reasoning for Security Protocol Analysis (FCS-ARSPA)*, 2007.
[11] T. Claverie, G. Avoine, S. Delaune, and J. Lopes-Esteves. Tamarin-based analysis of bluetooth uncovers two practical pairing confusion attacks. In *ESORICS 2023, Part III*, pages 100–119, 2023.
[12] T. Claverie and J. L. Esteves. Bluemirror: Reflections on bluetooth pairing and provisioning protocols. In *2021 IEEE Security and Privacy Workshops (SPW)*, pages 339–351, 2021.
[13] M. Fischlin and O. Sanina. Cryptographic analysis of the bluetooth secure connection protocol suite. In *ASIACRYPT 2021, Part II*, pages 696–725, 2021.
[14] M. Fischlin and O. Sanina. Fake it till you make it: Enhancing security of bluetooth secure connections via deferrable authentication. Cryptology ePrint Archive, Paper 2024/874, 2024. https://eprint.iacr.org/2024/874.
[15] K. Haataja and P. Toivanen. Two practical man-in-the-middle attacks on bluetooth secure simple pairing and countermeasures. *IEEE Transactions on Wireless Communications*, 9(1):384–392, 2010.
[16] K. Hypponen and K. M. Haataja. "nino" man-in-the-middle attack on bluetooth secure simple pairing. In *2007 3rd IEEE/IFIP International Conference in Central Asia on Internet*, pages 1–5, 2007.
[17] T. Jager, F. Kohlar, S. Schäge, and J. Schwenk. Generic compilers for authenticated key exchange. In *ASIACRYPT 2010*, pages 232–249, 2010.
[18] M. Jakobsson and S. Wetzel. Security weaknesses in Bluetooth. In *CT-RSA 2001*, pages 176–191, 2001.
[19] M. K. Jangid, Y. Zhang, and Z. Lin. Extrapolating formal analysis to uncover attacks in bluetooth passkey entry pairing. *NDSS 2023*, 2023.
[20] C. Kaufman, P. E. Hoffman, Y. Nir, P. Eronen, and T. Kivinen. Internet Key Exchange Protocol Version 2 (IKEv2). RFC 7296, 2014.
[21] H. Krawczyk. A unilateral-to-mutual authentication compiler for key exchange (with applications to client authentication in TLS 1.3). In *ACM CCS 2016*, pages 1438–1450, 2016.
[22] A. Levi, E. Çetintaş, M. Aydos, Ç. K. Koç, and M. U. Çağlayan. Relay attacks on bluetooth authentication and solutions. In *Computer and Information Sciences - ISCIS 2004*, pages 278–288, 2004.
[23] A. Y. Lindell. Comparison-based key exchange and the security of the numeric comparison mode in Bluetooth v2.1. In *CT-RSA 2009*, pages 66–83, 2009.
[24] T. Rosa. Bypassing passkey authentication in Bluetooth low energy. Cryptology ePrint Archive, Report 2013/309, 2013. https://eprint.iacr.org/2013/309.
[25] M. Ryan. Bluetooth: With low energy comes low security. In *7th USENIX Workshop on Offensive Technologies (WOOT 13)*, 2013.
[26] S. Schäge, J. Schwenk, and S. Lauer. Privacy-preserving authenticated key exchange and the case of IKEv2. In *PKC 2020, Part II*, pages 567–596, 2020.
[27] M. Shi, J. Chen, K. He, H. Zhao, M. Jia, and R. Du. Formal analysis and patching of BLE-SC pairing. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 37–52, 2023.
[28] J. H. Silverman. *Algorithmic Aspects of Elliptic Curves*, pages 363–408. Springer New York, New York, NY, 2009.
[29] D.-Z. Sun, Y. Mu, and W. Susilo. Man-in-the-middle attacks on secure simple pairing in bluetooth standard v5.0 and its countermeasure. *Personal Ubiquitous Comput.*, 22(1):55–67, 2018.
[30] D.-Z. Sun and L. Sun. On secure simple pairing in bluetooth standard v5.0-part i: Authenticated link key security and its home automation and entertainment applications. *Sensors*, 19(5), 2019.
[31] M. Troncoso and B. Hale. The bluetooth CYBORG: Analysis of the full human-machine passkey entry AKE protocol. In *NDSS 2021*, 2021.
[32] M. von Tschirschnitz, L. Peuckert, F. Franzen, and J. Grossklags. Method confusion attack on bluetooth pairing. In *2021 IEEE Symposium on Security and Privacy*, pages 1332–1347, 2021.
[33] J. Wu, Y. Nan, V. Kumar, D. J. Tian, A. Bianchi, M. Payer, and D. Xu. BLESA: Spoofing attacks against reconnections in bluetooth low energy. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*, 2020.
[34] J. Wu, R. Wu, D. Xu, D. Tian, and A. Bianchi. Sok: The long journey of exploiting and defending the legacy of king harald bluetooth. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 27–27, 2024.
[35] J. Wu, R. Wu, D. Xu, D. J. Tian, and A. Bianchi. Formal model-driven discovery of bluetooth protocol design vulnerabilities. In *2022 IEEE Symposium on Security and Privacy*, pages 2285–2303, 2022.
[36] H. Yin. Security analysis of bluetooth secure simple pairing protocols with extended threat model. *Journal of Information Security and Applications*, 72:103385, 2023.
[37] Y. Zhang, J. Weng, R. Dey, Y. Jin, Z. Lin, and X. Fu. Breaking secure pairing of bluetooth low energy using downgrade attacks. In *USENIX Security 2020*, pages 37–54, 2020.